

COMPUTER NETWORKS

Week-7

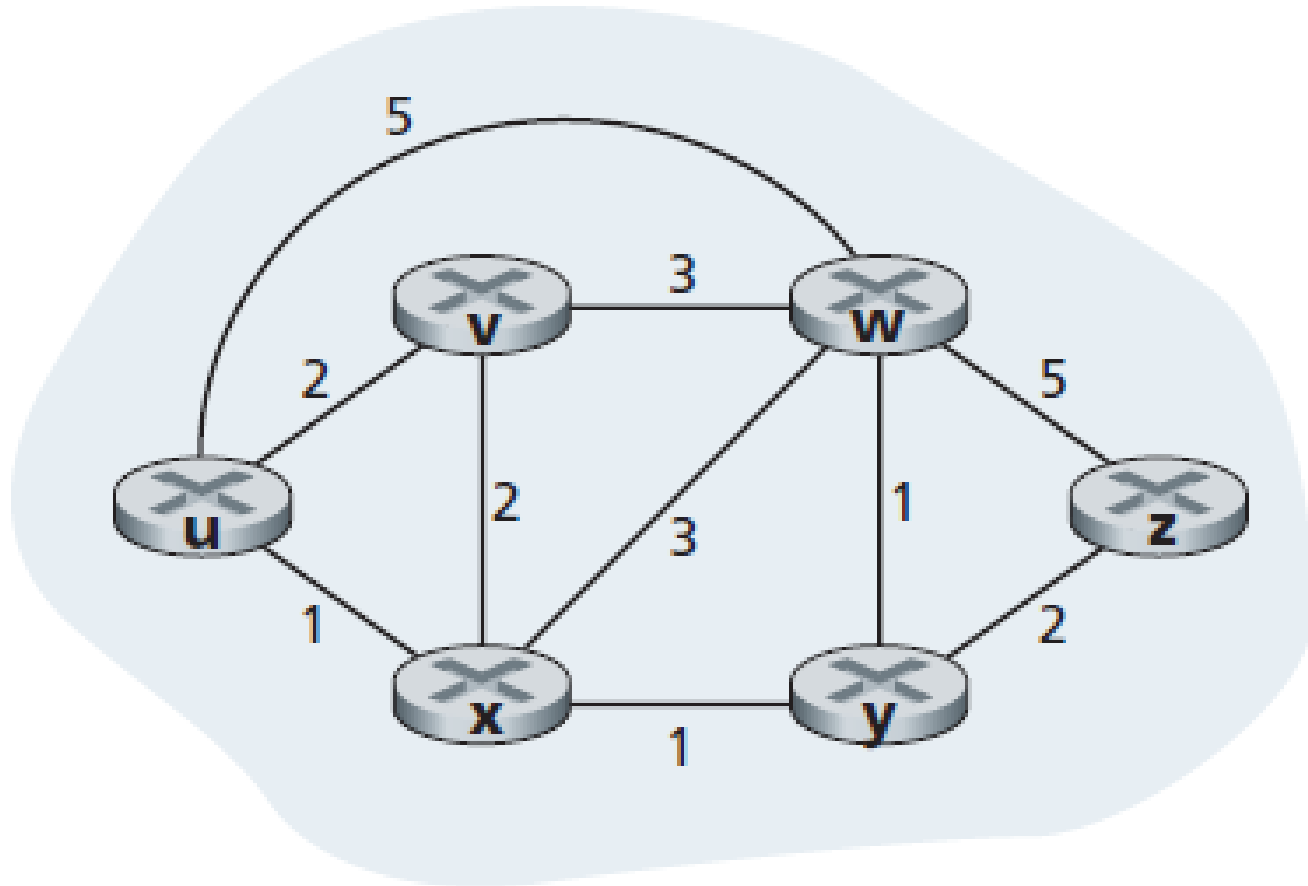
Routing Algorithms

- A **routing algorithm** → to determine good paths (equivalently, routes), from senders to receivers, through the network of routers
- A “**good**” path → one that has the least cost
- The routing algorithms that compute these paths are of fundamental importance

Graph

- A graph is used to formulate routing problems
- A **graph** $G = (N, E) \rightarrow$ a set N of nodes and a collection E of edges
- Each edge is a pair of nodes from N
- In the context of network-layer routing, the nodes in the graph represent Routers
- Routers $\rightarrow$ the points at which packet-forwarding decisions are made
- Edges $\rightarrow$ connect these nodes and represent the physical links between these routers

Abstract graph model of a network



Graph model

- An edge also has a value representing its **cost**
- Typically, an edge's cost may reflect the physical length of the corresponding link, the link speed, or the monetary cost associated with a link
- Lets not worry about how they are determined
- For any edge (x, y) in E , we denote $c(x, y)$ as the cost of the edge between nodes x and y
- If the pair (x, y) does not belong to E , we set $c(x, y) = \infty$
- Also note that we'll only consider undirected graphs (i.e., graphs whose edges do not have a direction)
- $c(x, y) = c(y, x)$
- However, in an algorithm cost may be considered different in different direction
- A node y is said to be a **neighbor** of node x if (x, y) belongs to E

Goal of an Algorithm

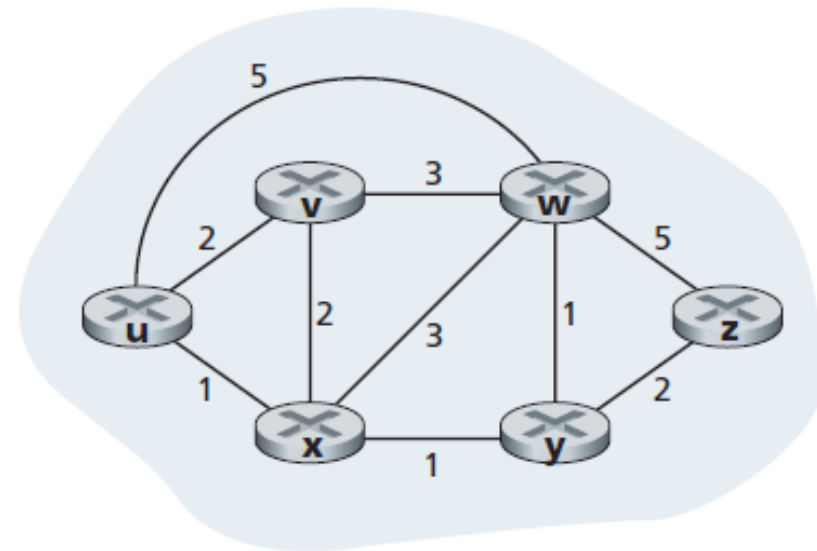
- Goal of a routing algorithm is to identify the least costly paths between sources and destinations
- A **path** in a graph $G = (N, E)$ is a sequence of nodes
- The cost of a path is simply the sum of all the edge costs along the path

Finding cost of a path

- Given any two nodes x and y , there are typically many paths between the two nodes
- Each path has a particular cost
- One or more of these paths is a **least-cost path**
- The **least-cost problem** → Find a path between the source and destination that has least cost

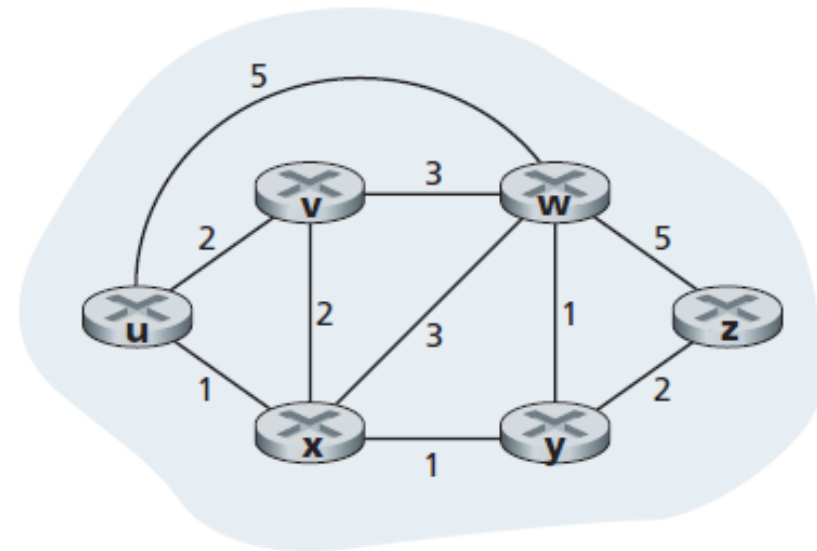
Finding cost of a path

- The least-cost path between source node *u* and destination node *w* ??



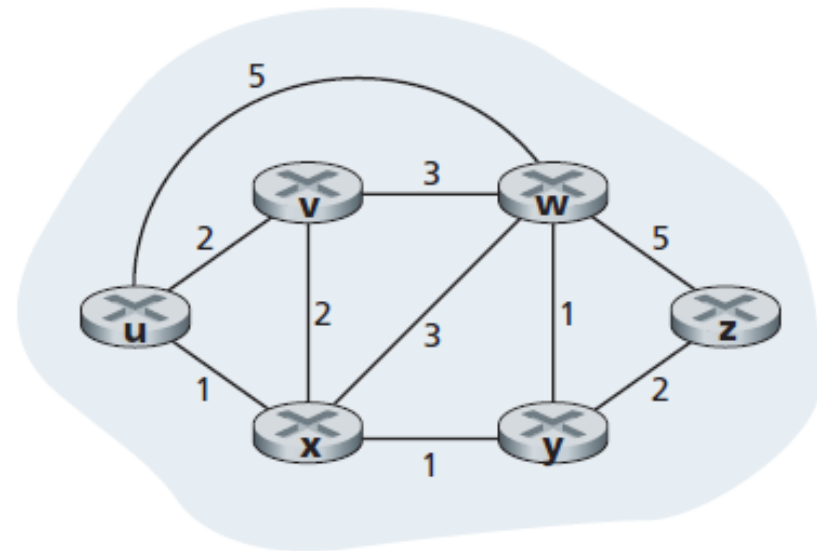
Finding cost of a path

- The least-cost path between source node *u* and destination node *w* ??
- $\rightarrow (u, x, y, w)$
- Path Cost ?



Finding cost of a path

- The least-cost path between source node *u* and destination node *w* ??
- $\rightarrow (u, x, y, w)$
- Path Cost = 3

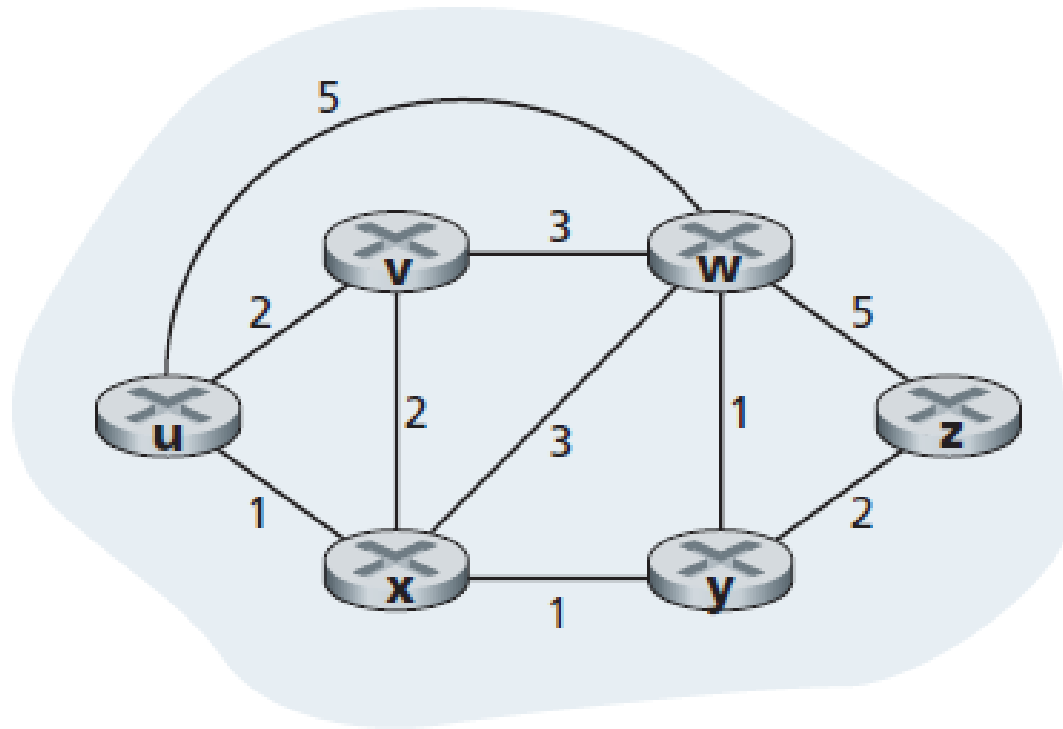


Shortest Path

- If all edges in the graph have the same cost
- The least cost path is also the **shortest path** (that is, the path with the smallest number of links between the source and the destination)

Ex-1

- Find the least-cost path from node u to z



Ex-1

- Find the least-cost path from node u to z in
- Did you check all of the 17 possible paths between u and z ?
- Probably not!
- Such a calculation is an example of a **centralized** routing algorithm—the routing algorithm was run in one location
- →your brain

Centralized routing algorithm

- A **centralized routing algorithm** computes the least-cost path between a source and destination using complete, global knowledge about the network
- The algorithm takes the connectivity between all nodes and all link costs as inputs
- Algorithms with global state information are often referred to as **link-state (LS) algorithms**
- These must be aware of the cost of each link in the network

Centralized routing algorithm

- A centralized routing algorithm relies on a single entity (or a few controllers) that has complete knowledge of the network and computes routes for all nodes
- All routers send their network information (topology, link costs) to a central controller
- The controller computes the best paths (often using algorithms like Dijkstra's Algorithm)
- Routes are distributed back to routers

Centralized routing algorithm

- For example
- Link-State Routing (e.g. OSPF)
- Open Shortest Path First (OSPF) is a link-state interior gateway routing protocol designed for IP networks
- OSPF is called an *Interior Gateway Routing Protocol (IGP)* because it is specifically designed to route traffic *within a single autonomous system*
- Does not handle routing policies between organizations
- An Autonomous System (AS) is a network under one administrative control (e.g., a university, company, or ISP)

Centralized routing algorithm

Advantages

- Accurate routing decisions (global view)
- Optimal paths are usually found

Disadvantages

- Single point of failure
- High overhead (needs full network info)
- Less scalable for very large networks

Decentralized routing algorithm

- The calculation of the least-cost path is carried out in an iterative, distributed manner by the routers
- No node has complete information about the costs of all network links
- Each node begins with only the knowledge of the costs of its own directly attached links through an iterative process of calculation and exchange of information with its neighboring nodes
- Each node gradually calculates the least-cost path to a destination or set of destinations
- E.g. distance-vector (DV) algorithm

Decentralized routing algorithm

- Allows each router to make decisions independently, based only on local information from neighbors
- Routers exchange information only with **neighbors**
- Each router updates its routing table **iteratively**
- **No** global knowledge of the entire network
- Distance Vector Routing (e.g., RIP)

Decentralized routing algorithm

- The Routing Information Protocol (RIP) is a distance-vector routing protocol used to exchange routing information within small to medium-sized IP networks

Decentralized routing algorithm

- Advantages
 - No single point of failure
 - Scales better in large/distributed networks
 - Simpler to implement
- Disadvantages
 - Slower convergence
 - May suffer from routing loops
 - Problems like count-to-infinity

Decentralized routing algorithm

- **Count to infinity problem:**
- It occurs when routers continuously increase the hop count to a destination that is no longer reachable, instead of realizing quickly that the destination is down
- Routers keep “counting” upward (1, 2, 3, ...) toward infinity because they mistakenly believe a path still exists

Network Behavior
Routers rely on neighbors
Route becomes invalid
Keep increasing hop count
Slow convergence

Alternate Classification of algorithms

- According to whether they are static or dynamic
- In **static** routing algorithms, routes change very slowly overtime, often as a result of human intervention (for example, a human manually editing a link costs)
- **Dynamic** routing algorithms change the routing paths as the network traffic loads or topology change
- A dynamic algorithm can be run either **periodically** or in direct response to topology or link cost changes
- These algorithms are more responsive to network changes

Static Routing Algo.

- Routes are manually configured by the network administrator and remain fixed unless changed manually
- No automatic updates
- Routes are predefined
- Simple to understand and implement
- No routing protocol required
- Predictable behaviour

Static Routing Algo.

- DisAdv
- Not scalable for large networks
- No automatic adaptation to failures
- High administrative effort
- **For example** for a university lab which has:
 - 20-30 computers
 - 1 router connected to the internet
 - No frequent network changes
 - Can easily manage via static routing
 - **Router configuration will include instructions for all traffic → send to ISP router (default route)**

Static Routing example

- Military / banking systems where route changes must be controlled
- No automatic updates from unknown routers
- Reduces risk of routing attacks

Dynamic Routing Algo.

- Uses routing protocols like:
- **RIP** : The Routing Information Protocol (RIP) is a distance-vector routing protocol
- **OSPF** : Open Shortest Path First (OSPF) is a link-state routing protocol used to find the best path between routers within an Autonomous System (AS)
- **BGP** : Border Gateway Protocol constructs an autonomous systems graph based on the information exchanged between BGP routers
- Routers exchange information
- Routes adapt to network changes

Dynamic Routing Algo.

- Advantages
 - Automatically handles failures
 - Scalable for large networks
 - Less manual configuration
- Disadvantages
 - CPU in a router is used to compute and update routes → higher CPU and memory usage
 - More complex
 - Possible security risks
- dynamic routing like in Google Maps

Dynamic Routing Algo.

- CPU calculates:
- Shortest paths
- Best routes
- Cost/metrics

Dynamic Routing examples

- When you open a website like Google or YouTube, your data travels across multiple networks worldwide
- Your university network
- ISPs like PTCL → to ensure uninterrupted internet service
- Data centers and clouds (AWS and Google cloud etc.)
- 4G and 5G mobile networks (we switch between towers continuously)
- Multinational companies (international presence)

Dynamic Routing

- We use Dynamic routing because :
- Servers are constantly added/removed
- Load balancing is required
- Failures must be handled instantly
- Fastest available path to be calculated
- High availability to be ensured

Classifying routing algos.

- According to whether they are load sensitive or load-insensitive
- In a **load-sensitive algorithm**, link costs vary dynamically to reflect the current level of congestion in the underlying link
- A high cost → a link is currently congested
- A routing algorithm will tend to choose routes around such a congested link
- Today's Internet routing algorithms (such as RIP, OSPF, and BGP) are **load-insensitive**, as a link's cost does not explicitly reflect its current (or recent past) level of congestion

Today's Routing algos.

- Load Insensitive
- If routing decisions changed based on traffic load?
- What would happen???

Today's Routing algos.

- Load Sensitive
- If routing decisions changed based on traffic load?
- What would happen???
- Routes would keep changing rapidly
- routes constantly flipping $\rightarrow$ oscillations (constant switching: $A \rightarrow B$, $B \rightarrow A$..)
- Network becomes **unstable**
- **Anything else??**

Why not Load sensitive?

- To be load-sensitive, routers would need:
- Continuous measurement of traffic on all links
- Frequent updates to all routers (CPU and memory overheads)
- → huge control overhead and consumes bandwidth
- By the time load info is shared:
- It is already outdated
- Decisions are based on **old data**

Why not Load sensitive?

- Bad for:
- Video calls
- Streaming
- Real-time applications

Book Reference

- Chapter 5
- James Kurose, W. Ross